Conditionals

INTRODUCTION TO JAVA

Jim White
Java Developer

What is a conditional?

- Decisions are a part of life and software
  - Often, there's a condition associate with a decision
  - "If it is raining, then I will take an umbrella"

Programming Conditionals

- if-then statements or *conditionals* are part of any programming language
- Conditionals take the form of:
  - **if** [some condition] **then** [code to execute]

Java conditional (if-then) statements

- The syntax for Java's if-then is:

```
if (condition) {
  // Java statements to be executed if condition is true
}
```

- If the condition is `true`, then the statements in the `{ }` get executed
  - If the condition is `false`, then the statements do not get executed

Conditions

- The condition (in a conditional) is anything that evaluates to `true` or `false`:
  - a boolean variable (boolean x = true;)
  - a comparison operation (8 > 9)
  - a logical operation (true && false)
    - Covered later in this video

if-then example

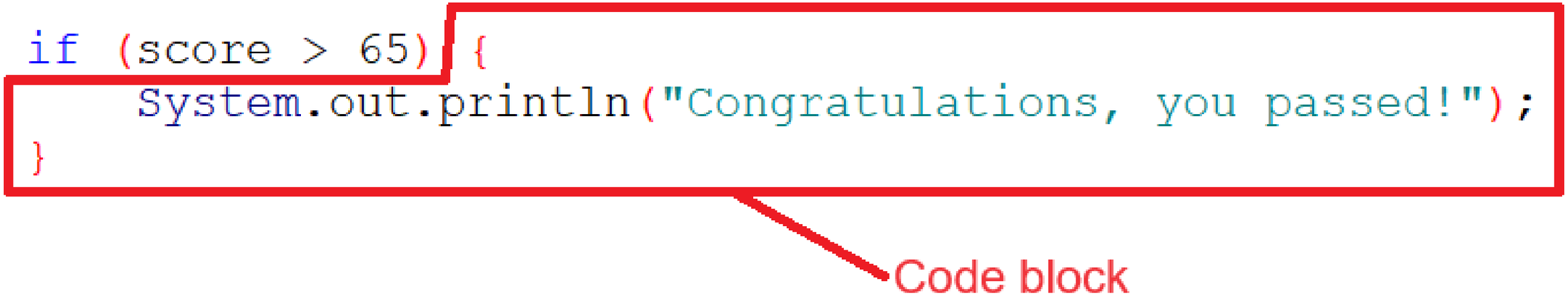
```
int score = 90;
if (score > 65) {
    System.out.println("Congratulations, you passed!");
}
```

- The condition (score > 65) is true
  - Triggers the execution of the statements between the `{ }`

```
Congratulations, you passed!
```

Code blocks

- The curly brackets `{ }` and the lines of code between them are called *code blocks*
  - Used with Java conditionals, loops and more
  - Groups statements together
  - The lines of a code block are all executed or all skipped



Code block variables

- Variables defined inside of a code block are only valid inside the code block
  - Using them outside the code block is not allowed

```
if (true) {
  int x = 9;
  x++;
}
System.out.println(x);  // Trying to use and print x here causes an error
```

```
error: cannot find symbol
        System.out.println(x);
                           ^
```

if-then-else

- Some actions/decisions are taken in the negative case of a conditional.

> If it is raining, then I will take an umbrella.
> But if it is not raining then I will take my sunglasses.

- Called the *else* of an if-then conditional
  - It is optional with if-then
  - The else code block is executed when the condition is `false`

```
if (condition) {
    // Statements to be executed if condition is true
} else {
    // Statements to be executed if condition is false
}
```

if-then-else example

```
int score = 55;
if (score > 65) {
    System.out.println("Congratulations, you passed!");
} else {
    System.out.println("Sorry, you did not pass the class.");
}
```

```
Sorry, you did not pass the class.
```

Optional code block

- Using a code block is optional when there is only one statement
  - For either the if-then or else blocks

```
if (score > 65)
    System.out.printly("Congratulations, you passed!");
else
    System.out.println("Sorry, you did not pass the class.");
```

Best practice and common mistake

- Not using a code block is considered poor practice
  - Adding more lines of code to if-then or else happens often
  - Developers can easily overlook missing `{ }`
- The last line of the code below would always be executed
  - The last line looks like its part of else code block but is not

```
if (score > 65)
  System.out.printly("Congratulations, you passed!");
else
  System.out.println("Sorry, you did not pass the class.");
  System.out.pritnln("Good luck on your text try");  // This line will always execute
```

- Not using code blocks can hide bugs like this one

Multiple if-then-else

- In some decisions, there are multiple conditions to explore

> If it is raining, then I will take an umbrella,
> ... or if it is snowing then I will take my scarf,
> ... or if it is sunny then I will take my sun glasses

```
if (condition1) {
  // Statements to be executed if condition1 is true
} else if (condition2) {
  // Statements to be executed if condition1 is false but condition2 is true
} else {
  // Statements to be executed if condition1 and condition2 are false
}
```

Multiple if-then-else example

- Use `if` and `else if` for multiple conditions
  - But still execute only one code block

```
int score = 85;
if (score > 90) {
    System.out.println("You passed the class and did outstanding.");
} else if (score > 80) {
    System.out.println("You passed the class.");
} else if (score > 65) {
    System.out.println("You passed the class, but need more practice.");
} else {
    System.out.println("Sorry, you did not pass the class.");
}
```

```
You passed the class.
```

Logical operators

- Logical operators `&&` and `||` are used between
  - boolean variables
  - booleans,
  - or operation that evaluates to boolean
- Use of a logical operator results in another boolean

Operator Example	Result
true && true	true
true && false	false
false && true	false
false && false	false
true || true	true
true || false	true
false || true	true
false || false	false

Logical operators in conditions

- Logical operators are used when there are multiple conditions
  - Logical operators help avoid multiple, nested conditionals

```
int a = 14;
if (a > 10) && (a < 20) {
    System.out.println ("Size is average");
} else if (a == 10) || (a == 20) {
    System.out.println ("Size is on the border");
} else {
    System.out.println ("Size is abnormal");
}
```

```
Size is average
```

Not operator

- `!` (the *not* operator) is used with a single boolean variable, value, or operation
  - Returns the opposite boolean

Operator Example	Result
!true	false
!false	true

```
System.out.println(!(5 < 6)); // Outputs false
System.out.println(!(5 > 6)); // Outputs true
```

Control flow

- Conditionals are part of the *control flow* of an application
- Control flow is the order of lines of code (statements) execution
- Control flow determines:
  - What code gets executed
  - Under what circumstances
  - How often

Let's practice!

INTRODUCTION TO JAVA

Looping

INTRODUCTION TO JAVA

Jim White
Java Developer

What are loops?

- Another common programming control flow mechanism, happens in life too
  - Based on some condition, repeatedly performing some action or actions
    > While hungry and thirsty, we continue to eat food and drink water
- In programming, *loop* control flow executes a group of statements over and over
  - Until some condition is met

Java looping

- Java has many looping control flow mechanisms
  - for loop
  - while loop
  - do-while loop
  - foreach loop (used with arrays and other collecting data structures)

For loop

- Repeat the execution of a code block a specified number of times
  - Useful when you know how many times you want to execute a code block
  - The number of executions is based on a counter

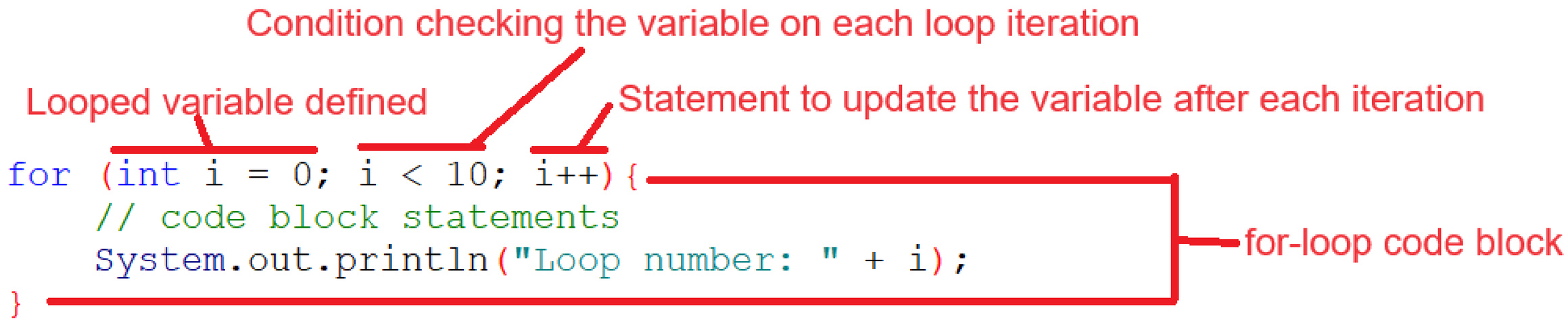
For loop syntax

```
for (variable defined; condition; variable update) {
    // Statements to be executed over and over
}
```

- A code block specifies what work is done each loop or *iteration*
- A for-loop requires a variable counter
  - A condition checks the counter and determine when to stop
  - A statement updates the counter
    - Often uses the increment statement to update the counter

For loop example

- Inside the for loop parenthesis:
  - counter variable
  - condition (checks the variable)
  - update statement (changes the variable)



How for works

```
for (int i = 0; i < 10; i++){
    // code block statements
    System.out.println("Loop number: " + i);
}
```

When encountering a for loop, Java:

- A) initializes the for loop variable (`i`) to an assigned value
- B) checks the condition (`i < 10`) - if false, stops looping
- C) executes the lines of code in the for-loop's code block
- D) updates the variable per the update statement formula (`i++`)
- E) returns to step B until the condition is false

While loop

- Executes a code block so long as (while) some condition is true

```
while (condition) {
  // Statements to execute so long as condition is true
}
```

- While loops have a condition
  - Checked before each iteration of the code block
  - If the condition is true, execute the code block
  - If the condition is false, stop executing
- Has no built in variables, counters, update statements
  - Just a conditional checked once per iteration.
- If the condition is false to start, the code block will never execute

While loop example

```
int i = 5;
while (i < 100) {
  i = i * 2;
  System.out.println("Looping ...");
}
```

```
Looping ...
Looping ...
Looping ...
Looping ...
Looping ...
```

Do while

- Similar to the while loop
  - Checks its condition after, not before, executing the code block
  - The code block is guaranteed to execute at least once

```
do  {
    // Statements to execute so long as condition is true
} while (condition);
```

Do while example

```
int i = 5;
do {
  i = i * 2;
  System.out.println("Looping...");
} while (i < 100);
```

```
Looping...
Looping...
Looping...
Looping...
Looping...
```

For each

- Has a syntax similar to the for loop
  - They are called "**for each**" but just use `for` in the syntax
  - Loops once for each element in a specified array

```
for (array_type array_element_variable_defined: array_variable) {
  // Statements to be executed over and over
}
```

- The loop defines a variable to hold each element of the array
  - If the array was empty, then the loop would not execute the code block
- Typically, the code block uses each array elements to do something

For each example

```
int[] prices = {6, 4, 3, 7};
int sum = 0;
for(int price: prices){
  sum = sum + price;
}
System.out.println(sum);
```

```
20
```

Infinite loop

- Each loop has some condition
  - That condition must be false to stop looping
- It's possible to set up a loop where the condition is never false

```
int i = 9
while (i < 10) {  // i is never changed in the loop
    System.out.println("Counting down: " + i);
}  // This loop will try to run forever
```

- This is called an *infinite loop*

Let's practice!

INTRODUCTION TO JAVA

Switch

INTRODUCTION TO JAVA

Jim White
Java Developer

What is a switch

- Switch serves a purpose similar to conditionals
  - Does some actions based on some condition
  - Used in place of multiple if-then-else conditionals
- Considered more concise than multiple if-then-else
- Switch also known as a *case* statement

Switch syntax

```
switch (expression) {
    case value1:  // if (expression == value1)
        // statements
        break;
    case value2:  // if (expression == value2)
        // statements
        break;
    // use as many case statements as needed
}
```

Switch expression

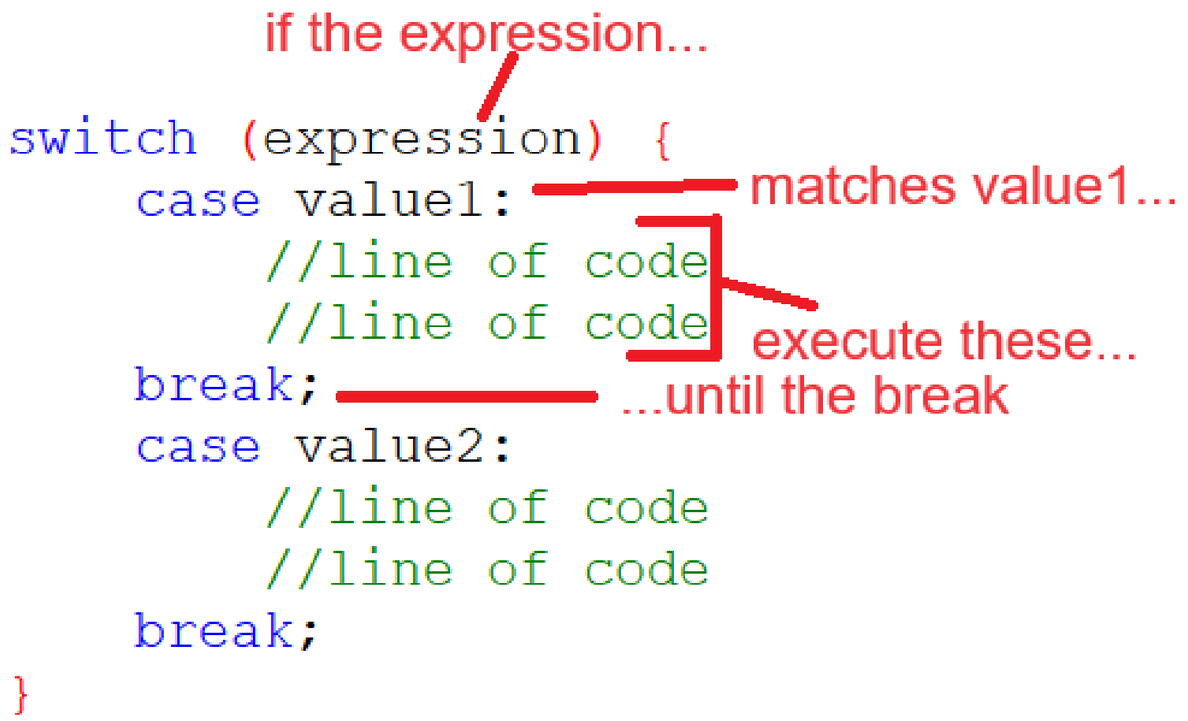
- The expression must evaluate to a single value
  - The value can only be of certain type:
    - byte, short, int, char, and String
- The expression could be
  - A variable
  - Anything that evaluates to the switch types

```
int i = 5;
switch (i) {
  // case statements
}
```

```
short x = 3;
short y = 4;
switch (x+y) {
  // case statements
}
```

Switch case statements

- Inside the switch curly brackets (`{ }`) are *cases*
  - A `case` value is compared to the expression
  - Each case value must be unique
  - When case value matches expression, execute the case statements
  - `break;` ends the case



Switch example

- Java first evaluates the switch expression
  - Here, the value of the `direction` variable
- Compares the expression value to each case value
  - From top to bottom, looking for a match
  - Matching the 3rd case in this example
- Executes the statements under the case that matches
  - `break` signals the end of the case statements

```
char direction = 'N';
switch (direction) {
  case 'E':
    System.out.println("Go east");
    break;
  case 'W':
    System.out.println("Go west");
    break;
  case 'N':
    System.out.println("Go north");
    break;
  case 'S':
    System.out.println("Go south");
    break;
}
```

```
Go north
```

Comparing switch and conditionals

- Conditionals could be used to complete the same task
  - Here is conditional code to do the same work

```
char direction = 'N';
if (direction == 'E') {
    System.out.println("Go east");
} else if (direction =='W') {
    System.out.println("Go west");
} else if (direction == 'N') {
    System.out.println("Go north");
} else if (direction == 'S') {
    System.out.println("Go south");
}
```

Cases without breaks

```
int x = 2;
switch (x) {
  case 1:
    System.out.println("expression is 1");
    break;
  case 2:
  case 3:
  case 4:
    System.out.println("print this if x is 2, 3, or 4");
    break;
  case 5:
    System.out.println("expression is 5");
    break;
}
```

```
print this if x is 2, 3, or 4
```

Break

- `break` is optional
  - Without break, Java execute statements in the next case until it hits a break
- Removing `break` allows for combining multiple cases to use the same statements

Default

- An optional `default` case can be added to the end of a switch
  - The default case has no value to compare to the expression
  - If no case value matches the expression, the default statements get executed
  - `default` must be the last case in a switch
- `default` is a way to have an "all other values" case

Default example

```
char direction = 'Z';
switch (direction) {
  case 'E':
    System.out.println("Go east");
    break;
  case 'W':
    System.out.println("Go west");
    break;
  case 'N':
    System.out.println("Go north");
    break;
  case 'S':
    System.out.println("Go south");
    break;
  default:
    System.out.println("We are lost");
}
```

```
We are lost
```

Let's practice!

INTRODUCTION TO JAVA

Exception Handling

INTRODUCTION TO JAVA

Jim White
Java Developer

What is an exception?

- Problems or issues can occur in applications
  - Due to wrong input
  - Due to coding errors
  - Due to unforeseen circumstances
- Problems or issues are called *exceptions*
  - Interrupt the normal control flow
- Java *throws* exceptions

What is exception handling

- Applications can watch for exceptions
- Applications can then *handle* exceptions
  - Allowing applications to survive issues and problems
  - Potentially allowing for return to normal operation

Exception vs error

- *Errors* are serious issues or problems
- Applications **cannot** handle and recover from errors
  - Errors cause the application to "crash" or stop
  - Example: an out of memory problem is an error in Java
- Applications can handle and recover from exceptions
  - Try to access elements outside the array size is an exception.

Checked vs runtime exceptions

- *Checked* exceptions are watched and handled
  - More common issues/problems
  - Issues often caused by outside influences
  - Ex: file not found
- *Runtime* (*unchecked*) exceptions are optionally watched and handled
  - Less common issues/problems
  - Issues often cause by developer
  - Ex: accessing an array index that does not exist
  - Application fails when runtime exception not handled

Try

- Use `try` in front of a code block to watch for exceptions

```
try {
    // Statements to be executed and watched for exceptions
}
```

Catch

- Use `catch` to define what exceptions to look for
  - Use `Exception` to watch for all types of exception
  - Define an exception variable to hold the exception detected
- Use a code block to define what happens if the exception does occur

```
try {
    // Statements to be executed and watched for exceptions
} catch (Exception variable) {
    // Statements to execute if an exception has occurred
}
```

Try/Catch example

```
int x = 7;
int y = 0;
try {
    System.out.println(x/y);
} catch (Exception ex){
    System.out.println("Did you try to divide by zero?");
}
```

```
Did you try to divide by zero?
```

Try with multiple catch

```
int x = 7;
int y = 0;
try {
    System.out.println(x/y);
} catch (ArithmeticException ex1){
    System.out.println("Did you try to divide by zero?");
} catch (Exception ex2) {
    System.out.println("Ooops - something unexpected happened");
}
```

```
Did you try to divide by zero?
```

Finally

```
int x = 10;
int y = 5;
try {
 System.out.println(x/y);
} catch (ArithmeticException ex){
 System.out.println("Divide by zero?");
}  finally {
 System.out.println("Clean up here");
}
```

```
1
Clean up occurs here
```

```
int x = 10;
int y = 0;
try {
 System.out.println(x/y);
} catch (ArithmeticException ex){
 System.out.println("Divide by zero?");
}  finally {
 System.out.println("Clean up here");
}
```

```
Divide by zero?
Clean up here
```

Let's practice!

INTRODUCTION TO JAVA

Wrap-up

INTRODUCTION TO JAVA

Jim White
Java Developer

Recap

What we learned:

- What is Java and why its an important programming language
- How to create Java applications using `class` and `method`s
- Java primitives and primitive operators
- Creating and using variables
- What, when and why around Strings and Arrays
- How to create and use Strings
- How to create and use Arrays
- Java syntax to include conditionals, loops and switches
- And the basics of error handling

What's next?

- Internet Sources
  - **Baeldung**: website dedicated to Java topics and assistance
  - **Oracle's Java site**: the stewards of Java technology (Oracle) web site
  - **dev.java**: Java developer news, events, and other information
  - **Open JDK**: community dedicated to a free and open-source implementation of Java
- Books
  - *Head First Java* by Sierra et al
  - *Learn Java In One Day* by Jamie Chan
  - *Java for Beginners* by Brady Ellison
- Additional online course from **DataCamp**
  - **Introduction to Object-Oriented Programming in Java**

Congratulations!

INTRODUCTION TO JAVA